Temporal Subset Approximate Nearest-Neighbor Search

A Workload-Aware Comparison of Global, Partitioned, and Hybrid Strategies

Scott Faust
University of Nebraska Omaha
Omaha, Nebraska, USA
ScottFaust@UNOmaha.com

Abstract

Nearest-neighbor search in high-dimensional data has evolved from exact geometric indexes to approximate nearest-neighbor (ANN) systems that trade exactness for scalability. That literature is now mature for unconstrained similarity search, but many practical systems must answer nearest-neighbor queries over a changing subset of the data rather than over the full collection. This report studies a temporal subset ANN setting in which each record has interval validity, scalar metadata, and optional categorical structure, and each query asks for the top-k nearest neighbors within a time window and filter predicate.

Building on the literature on k-d trees, R-trees, locality-sensitive hashing, product quantization, graph-based search, and large-scale ANN systems, this report develops and evaluates a research prototype with four strategies: an exact filtered oracle, a global ANN plus post-filter baseline, a partition-first ANN method, and a workload-aware hybrid planner. Experiments on synthetic workloads at $n = 5000$ with 32- and 64-dimensional vectors show a clear pattern. Global post-filter ANN performs poorly under subset constraints, with mean Recall@10 ranging from approximately 0.30 to 0.69. Partition-first ANN remains the strongest fixed strategy, achieving Recall@10 of 1.0 across the tested workloads while also producing the lowest median latency. The hybrid planner improves substantially over the global baseline and preserves high recall, but planner match rates of roughly 0.27 to 0.50 and nontrivial latency regret show that adaptive decision quality remains the main unresolved problem. The resulting conclusion is that temporal subset ANN is best treated as a routing and workload-selection problem rather than as ordinary ANN followed by filtering.

Keywords

approximate nearest neighbor, similarity search, temporal filtering, vector search, indexing structures, high-dimensional data

1 Introduction

High-dimensional nearest-neighbor and similarity search is a core problem in algorithms, data structures, and modern intelligent systems. It appears in image retrieval, document search, recommender systems, embedding databases, and many forms of machine learning inference. Classical multidimensional indexes such as the k-d tree and the R-tree established the foundations of geometric search by organizing data so that large portions of the search space could be excluded without direct comparison [2, 6]. As dimensions rise, however, the geometric selectivity that makes exact pruning effective weakens dramatically, and much of the literature shifts toward approximate nearest-neighbor methods that accept bounded error in exchange for practical speed [1, 5, 7].

The literature on ANN is now rich and varied. Hash-based methods use locality-sensitive hashing to preserve similarity probabilistically [5, 7]. Tree-based practical systems such as FLANN use randomized forests and clustered trees, and intrinsic-dimension approaches such as cover trees seek structure beyond the raw ambient dimension [3, 11]. Compression-based methods such as product quantization reduce memory and comparison costs [8]. Graph-based systems such as NSG and HNSW make search itself a traversal problem over approximate neighborhood graphs [4, 10]. GPU-oriented systems further show that large-scale performance depends not only on index structure but on hardware-aware kernels and memory behavior [9].

This report investigates a narrower but practically important variant of the problem: nearest-neighbor search over a *temporal subset*. In this setting, the query does not search the entire vector collection. Instead, it searches only records whose validity interval intersects a requested time window and whose metadata satisfy a scalar constraint. This changes the design problem substantially. A global ANN index can still provide candidate neighbors, but many of those candidates may be invalid under the temporal filter. Conversely, a partitioned search structure can route queries more precisely, but may pay maintenance cost when data expire or become inactive.

The central research question of this report is therefore:

> How should ANN search be organized when nearest-neighbor queries must be answered over dynamic temporal subsets rather than over the full high-dimensional collection?

The report contributes three things. First, it synthesizes the broader ANN literature into a framework that motivates temporal subset search as a natural next problem rather than as a separate topic. Second, it presents a prototype system with four strategies: exact filtered search, global ANN plus post-filtering, partition-first ANN, and a hybrid planner. Third, it evaluates those strategies under multiple temporal workloads and shows that partition-first routing is the strongest fixed method in the current prototype, while adaptive hybrid planning remains promising but incomplete.

2 Background and Related Work

2.1 Exact multidimensional indexing

The earliest influential multidimensional indexes were designed for exact search. Bentley's k-d tree recursively partitions data along coordinate axes and generalizes binary search tree ideas to multiple dimensions [2]. Guttman's R-tree organizes spatial objects with bounding rectangles and became especially important in multidimensional database systems [6]. Both methods rely on the same

fundamental idea: efficient search comes from structured exclusion of regions that cannot contain the answer.

These structures are elegant and foundational, but they do not scale cleanly into high-dimensional settings. As dimensionality increases, partitions become less discriminative and bounding regions overlap more heavily. The consequence is that exact structures often degenerate toward scan-like behavior, which is one of the central motivations for later ANN work [1, 5, 7].

2.2 Approximate nearest-neighbor search

Indyk and Motwani formalized the approximate nearest-neighbor problem and showed that exactness could be relaxed while still preserving useful retrieval guarantees [7]. Their locality-sensitive hashing framework made similarity search sublinear in settings where exact geometric methods were already breaking down. Gionis, Indyk, and Motwani then connected this idea more directly to similarity search in high-dimensional datasets [5].

Later work diversified the design space. FLANN demonstrated that practical ANN performance depends strongly on data characteristics and that automated method selection is valuable even among tree-based methods [11]. Cover trees showed that intrinsic structure can matter more than nominal dimension in some metric spaces [3]. Product quantization reframed large-scale ANN partly as a compression problem [8]. NSG and HNSW showed that graph navigation can provide excellent empirical recall-latency tradeoffs [4, 10]. At billion scale, Johnson, Douze, and Jégou further showed that hardware-aware system design is inseparable from ANN performance [9].

2.3 Why temporal subset search is a distinct problem

These families of methods largely assume that the search space is the full vector collection. In many real systems, however, a user rarely wants the nearest vector across everything. Instead, the user wants the nearest vector among items that are currently valid, fall into a date range, satisfy a scalar constraint, or belong to a category. In that setting, the algorithm must handle two intertwined problems:

(1) efficient similarity search in high dimensions, and
(2) efficient restriction of that search to a changing subset.

This is not a trivial wrapper around ANN. A global index may retrieve high-quality neighbors that are later rejected by the filter, wasting search effort. A partitioned structure may reduce that waste, but only if the partitions align well with the subset constraints and if maintenance under insertions, expirations, and open-ended intervals remains manageable. Temporal subset ANN therefore sits at the intersection of indexing, filtering, and dynamic maintenance.

3 Problem Formulation

Each record in the prototype has the form

$$r_i = (v_i, t_i^{start}, t_i^{end}, p_i, c_i),$$

where $v_i \in \mathbb{R}^d$ is the vector embedding, t_i^{start} and t_i^{end} define the record's validity interval, p_i is a scalar price-like attribute, and c_i is an optional category. Open-ended validity is represented by an unset upper endpoint.

Each query has the form

$$q = (x, [T_s, T_e], [P_s, P_e], c, k),$$

where x is the query vector, $[T_s, T_e]$ is the requested time window, $[P_s, P_e]$ is the requested scalar range, c is an optional category restriction, and k is the number of neighbors requested. A record is eligible when its validity interval intersects the query window and its metadata satisfy the scalar and categorical constraints.

The algorithmic goal is to return the top-k nearest eligible records under Euclidean distance while minimizing latency and maintenance overhead. This report evaluates not only retrieval quality but also the planner's ability to select an effective strategy under changing subset conditions.

4 Prototype Design

4.1 Algorithms

The prototype compares four methods.

Exact filtered oracle. The oracle performs a flat scan over the currently active records, applies all filters exactly, computes the exact distances of the surviving candidates, and returns the true top-k. This method is slowest asymptotically, but it provides the correctness baseline.

Global ANN then filter. This baseline builds a single ANN index over the active vectors and retrieves approximate nearest neighbors before filtering them by temporal validity and metadata. The method is conceptually simple, but it is expected to degrade when many top ANN candidates fall outside the valid subset.

Partition-first ANN.. The partitioned method routes records by time and scalar buckets and maintains local ANN indexes inside sufficiently large cells. Small cells fall back to exact local scanning. The method also maintains interval-envelope metadata per cell, including the minimum and maximum interval endpoints, price bounds, open-ended counts, and category counts. These summaries are used to skip cells that cannot intersect the query.

Hybrid planner. The hybrid method estimates subset size from cell metadata and selects among exact search, partition-first ANN, and global ANN plus post-filtering. The current planner is rule-based rather than learned, but it logs its own feature values and outcomes so that planner quality can be analyzed directly.

4.2 Dynamic maintenance

The prototype supports insertions, logical deletions, and expirations. Records that become inactive are not immediately removed from every structure. Instead, the system uses tombstone ratios and threshold-triggered rebuilds. The global index compacts inactive records during rebuild, and the partitioned index compacts heavily tombstoned cells during local rebuilds. This makes the prototype more realistic than one that rebuilds the full structure after every update.

4.3 Implementation artifact

The implementation is organized as a reproducible Python artifact rather than only as pseudocode. The repository includes separate modules for the exact oracle, global post-filter index, partition-first

index, hybrid planner, workload generation, experiment execution, result summarization, plotting, and planner training. The ANN backend is also intentionally optional. When `hnswlib` is installed, the prototype can use an HNSW-backed candidate generator; when it is unavailable, the same interfaces fall back to brute-force candidate generation so that tests, smoke runs, and report-generation scripts remain executable in a minimal environment.

The experiment harness writes per-query CSV traces and run-level summaries. Each query row records latency, Recall@10, valid-result rate, exact subset size, subset-estimate error, planner mode, candidate counts, partitions touched, tombstone counts, deleted and expired counts, rebuild and compaction counts, expansion rounds, and a `best_mode` label. These traces are useful for two reasons. First, they make the reported aggregate figures auditable from query-level observations. Second, they provide the feature and label data needed for learned planner experiments. The artifact therefore includes an initial nearest-centroid planner workflow, although the main results in this report focus on the rule-based hybrid planner.

5 Experimental Methodology

5.1 Synthetic data and workloads

The experiments use synthetic vector data with controllable cluster structure, temporal span, scalar correlation, optional category structure, and interval lifetimes. The final experiments use $n = 5000$ records and evaluate four workload regimes:

(1) **Static**: no turnover.
(2) **Short-life expire**: many records expire quickly.
(3) **Long-life expire**: intervals are long-lived before expiration.
(4) **Open-ended heavy**: many records remain active indefinitely.

Each workload is evaluated in 32 and 64 dimensions. The reported summary statistics aggregate repeated runs over multiple seeds.

5.2 Metrics

The evaluation uses:

- Recall@10,
- valid-result rate,
- p50, p95, and p99 latency,
- subset-estimate error,
- rebuild and compaction counts,
- planner mode frequencies,
- planner match rate,
- planner latency regret, and
- planner recall gap.

Planner regret compares the hybrid method to the fastest non-hybrid method that satisfies a recall floor for the same query. This is important because it evaluates the *decision quality* of the hybrid planner, not only the raw retrieval quality.

6 Results

6.1 Fixed strategy comparison

The clearest result is that global ANN plus post-filtering is the weakest strategy in this temporal subset setting. Averaged over the two tested dimensions, the global method achieves mean Recall@10 values of roughly 0.31 in the static workload, 0.67 in short-life expire, 0.69 in long-life expire, and 0.67 in open-ended-heavy workloads. In contrast, partition-first ANN maintains Recall@10 of 1.0 in every workload tested. Figure 1 summarizes this pattern.

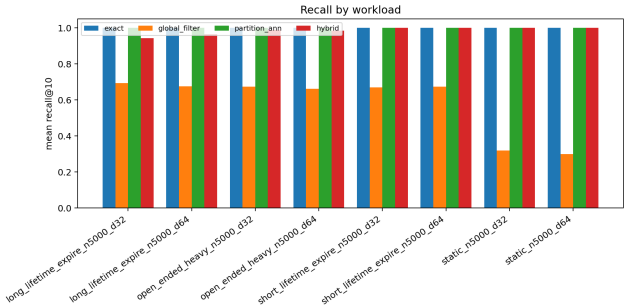

Figure 1: Mean Recall@10 averaged over 32- and 64-dimensional runs at $n = 5000$. Global ANN plus post-filtering degrades substantially under subset constraints, while partition-first ANN remains exact in the tested workloads.

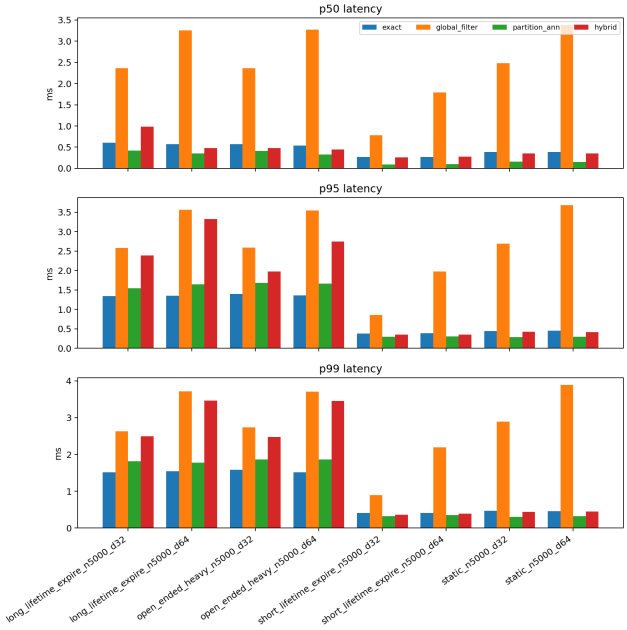

Figure 2: Mean p50 latency averaged over 32- and 64-dimensional runs at $n = 5000$. Partition-first ANN is the fastest fixed strategy in the current prototype.

Latency tells a similarly clear story. Partition-first ANN also produces the lowest median latency across the tested workloads, with average p50 latency between approximately 0.10 ms and 0.39 ms. Exact filtered search is slower but still competitive because the subset constraints reduce candidate counts in many cases. Global ANN plus post-filtering is much slower, often between roughly 1.3 ms and 2.9 ms on average. Figure 2 shows these differences.

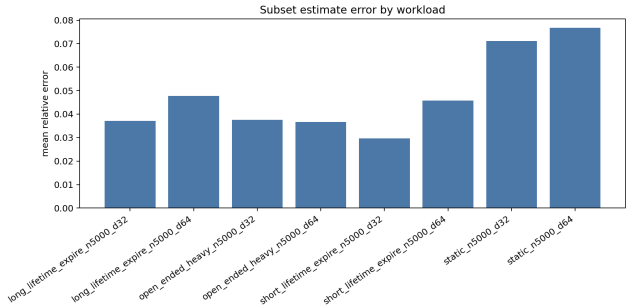

Figure 3: Mean subset-estimate error by workload. Estimation quality is now relatively stable, which shifts attention to planner quality rather than estimator failure.

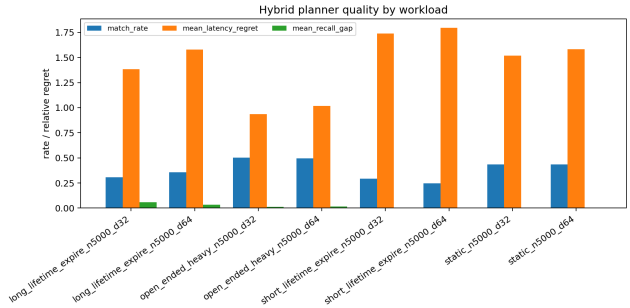

Figure 4: Hybrid planner quality by workload. Match rate remains modest and latency regret remains substantial, showing that planner selection is still the main unresolved problem.

Table 1 reports the workload-averaged Recall@10 and p50 latency across dimensions. The strongest fixed conclusion is straightforward: in this prototype, routing search into the relevant temporal and scalar partitions is substantially more effective than retrieving neighbors globally and filtering afterward.

6.2 Estimator behavior

A major improvement over earlier versions of the prototype is that subset estimation is no longer the dominant failure point. Mean subset-estimate error is modest across all reported workloads: roughly 0.07 to 0.08 in the static setting, about 0.03 to 0.05 in the short- and long-lifetime settings, and around 0.037 in the open-ended-heavy setting. Figure 3 visualizes these values.

This matters because it changes the interpretation of the hybrid results. Earlier development artifacts suggested that poor subset estimates might explain planner failures under hard temporal regimes. The final artifact indicates otherwise: estimator quality is now reasonably good. The remaining weakness lies in the planner's decision policy.

6.3 Hybrid planner behavior

The hybrid planner substantially improves over the global post-filter baseline. It keeps Recall@10 at 1.0 in the static and short-life workloads, reaches about 0.954 in the long-life workload, and about 0.987 in the open-ended-heavy workload. This is a meaningful improvement over global filtering, especially because valid-result rate remains 1.0 throughout the experiments, which indicates that the loss is due to missed neighbors rather than invalid returned records.

At the same time, the planner is still not choosing the best strategy often enough. Averaged across dimensions, hybrid planner match rate is about 0.43 in the static workload, 0.27 in short-life expire, 0.33 in long-life expire, and 0.50 in open-ended-heavy workloads. Mean planner latency regret ranges from about 0.98 to 1.77. Figure 4 summarizes these planner-quality metrics.

This is the most important negative result in the paper. The prototype now demonstrates that *adaptive search is worth attempting*, but it does not yet demonstrate that the current rule-based planner is close to optimal. The planner still overuses exact search in easy settings and overuses global filtering in some hard temporal settings. In other words, the design space is well motivated, but the current planner remains a baseline rather than a final solution.

6.4 Maintenance costs

Dynamic maintenance is visible in the reported rebuild and compaction statistics. Short-life and open-ended workloads generate large cell-rebuild counts and compaction events because those workloads create the heaviest temporal turnover. This is not surprising: the same conditions that make subset routing valuable also increase maintenance pressure. The key point is that the partition-first method still retains its retrieval advantage even under these maintenance-heavy workloads. That suggests the maintenance cost is real but not disqualifying in the current prototype.

7 Discussion

The experiments support three broader conclusions.

First, temporal subset ANN should not be treated as ordinary ANN followed by filtering. The poor recall of the global post-filter baseline shows that the filter changes the problem fundamentally rather than merely narrowing the final answer set. This is consistent with the larger ANN literature, which repeatedly shows that the success of an index depends on how well its structure matches the workload [9–11].

Second, partition-first routing is highly effective in the present prototype. This aligns with the intuition that when queries are defined over changing subsets, search should begin with the subset whenever possible rather than with the full collection. In this report's experiments, the partition-first strategy dominates both exact and global-filter methods as a fixed design choice.

Third, hybridization is promising but incomplete. The hybrid planner clearly improves over the global baseline and often preserves exact-level recall. However, planner match rates and regret show that selecting the right mode remains difficult. The current system therefore supports a nuanced conclusion: *the adaptive formulation is justified, but the planner itself remains the main research challenge.*

Table 1: Average Recall@10 and p50 latency (ms) across 32- and 64-dimensional runs at $n = 5000$.

Workload	Method	Recall@10	p50 latency (ms)
Static	Exact	1.000	0.389
Static	Global filter	0.309	2.927
Static	Hybrid	1.000	0.354
Static	Partition-first ANN	1.000	0.154
Short-life expire	Exact	1.000	0.269
Short-life expire	Global filter	0.672	1.287
Short-life expire	Hybrid	1.000	0.270
Short-life expire	Partition-first ANN	1.000	0.098
Long-life expire	Exact	1.000	0.585
Long-life expire	Global filter	0.685	2.805
Long-life expire	Hybrid	0.954	0.730
Long-life expire	Partition-first ANN	1.000	0.388
Open-ended heavy	Exact	1.000	0.555
Open-ended heavy	Global filter	0.669	2.816
Open-ended heavy	Hybrid	0.987	0.463
Open-ended heavy	Partition-first ANN	1.000	0.368

8 Limitations and Future Work

The report has several limitations. The experiments are synthetic rather than based on a production vector database, so real-world distributions may differ. The metadata model includes a single scalar filter and optional category rather than many heterogeneous attributes. The reported hybrid planner is rule-based, and the included nearest-centroid learned-planner workflow is only an initial artifact rather than a robust planner-selection solution. Finally, the current prototype measures dynamic maintenance through rebuild and compaction counts, but it does not yet optimize those operations aggressively.

The most direct next step is to replace the rule-based planner with a stronger learned planner trained on the logged query features and per-query outcomes. The final artifact already records the information needed for that step, including estimator error, mode choices, planner regret, and workload descriptors. A learned planner could then be evaluated against the rule-based planner using the same benchmark harness.

9 Conclusion

This report studied temporal subset ANN as an extension of the broader literature on high-dimensional indexing and approximate nearest-neighbor search. The literature shows a progression from exact multidimensional indexes such as k-d trees and R-trees to approximate methods based on hashing, trees, compression, graph navigation, and hardware-aware execution [2, 4–11]. The experimental contribution of this report is to show how those ideas behave once nearest-neighbor queries are constrained to dynamic temporal subsets.

The strongest empirical finding is that global ANN plus post-filtering is a poor fit for this problem, while partition-first ANN is consistently strong in the current prototype. The hybrid planner improves over the global baseline and demonstrates that adaptive strategy selection is worthwhile, but planner match rate and regret show that the current decision policy is still suboptimal. The final conclusion is therefore both practical and forward-looking: temporal subset ANN is best understood as a routing and workload-selection problem, and the next major research step is learning how to choose among otherwise strong search strategies.

References

[1] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. 2018. Approximate Nearest Neighbor Search in High Dimensions. *arXiv preprint arXiv:1806.09823* (2018).

[2] Jon Louis Bentley. 1975. Multidimensional Binary Search Trees Used for Associative Searching. *Commun. ACM* 18, 9 (1975), 509–517.

[3] Alina Beygelzimer, Sham Kakade, and John Langford. 2006. Cover Trees for Nearest Neighbor. In *Proceedings of the 23rd International Conference on Machine Learning*. 97–104.

[4] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474.

[5] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. 1999. Similarity Search in High Dimensions via Hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases*. 518–529.

[6] Antonin Guttman. 1984. R-Trees: A Dynamic Index Structure for Spatial Searching. In *Proceedings of the 1984 ACM SIGMOD International Conference on Management of Data*. 47–57.

[7] Piotr Indyk and Rajeev Motwani. 1998. Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*. 604–613.

[8] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[9] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[10] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[11] Marius Muja and David G. Lowe. 2014. Scalable Nearest Neighbor Algorithms for High Dimensional Data. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 11 (2014), 2227–2240.